

Optimizing Zero-Shot Model Compression Pipelines: A Progress Report

Henry Morgan
University of Washington Bothell
USA
hdmorgan@uw.edu

Abstract

Modern deep learning models are often too large and computationally expensive for deployment on resource-constrained edge devices. Model compression techniques such as quantization, pruning, and knowledge distillation are essential for optimizing these models. This project systematically explores the effects of combining these techniques in a zero-shot (data-free) setting. We have implemented a flexible pipeline that allows for the sequential application of various compression methods and evaluates their impact on model size, sparsity, and latency. This report details the progress made in implementing these compression techniques, the pipeline infrastructure, and initial experimental results. We also outline the plan for the final paper, which will involve more extensive experiments and analysis.

ACM Reference Format:

Henry Morgan. 2026. Optimizing Zero-Shot Model Compression Pipelines: A Progress Report. In . ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

The increasing size and complexity of state-of-the-art deep learning models present significant challenges for their deployment on edge devices with limited computational power and memory. Model compression aims to reduce the size and inference latency of these models while preserving their predictive accuracy. This project focuses on zero-shot compression techniques, which are particularly valuable as they do not require access to the original training data.

Our primary goal is to understand the interactions between different compression methods and to identify optimal orderings for their application. We have developed a modular framework in Python using PyTorch to construct and evaluate various compression pipelines.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

2 Project Progress

2.1 System Design and Implementation

We have designed and implemented a `CompressionPipeline` that orchestrates the application of multiple compression techniques. The core components of our system are:

- **BaseCompressor Interface:** An abstract base class that defines a common interface for all compression techniques. Each compressor implements a `compress` method that takes a model and returns a compressed model along with meta-data.
- **Compression Techniques:** We have implemented the following zero-shot compression techniques:
 - **Quantization:** `DynamicQuantizer` and `StaticQuantizer` to reduce model precision to INT8.
 - **Pruning:** `MagnitudePruner` for unstructured pruning and `StructuredPruner` for structured pruning of weights.
 - **Knowledge Distillation:** `ZeroShotDistiller` that synthesizes pseudo-inputs from the teacher model's batch normalization statistics.

The overall workflow is managed by the `run_experiment.py` script, which defines and executes different pipeline configurations, collects metrics, and presents a comparative summary.

2.2 Initial Experiments and Results

We conducted an initial set of experiments to compare different orderings of dynamic quantization and magnitude pruning. The results are summarized in Table 1. The key trade-offs between model size and inference latency are visualized in Figure 1 and Figure 2.

Table 1: Experiment Results Summary

Pipeline	Size (MB)	Sparsity (%)
Baseline (no compression)	0.39	0.00
Dynamic Quantization only	0.10	0.00
Magnitude Pruning (50%) only	0.39	50.00
Structured Pruning (30%) only	0.39	30.08
Prune → Quantize	0.10	0.00
Quantize → Prune	0.10	0.00
Structured Prune → Magnitude Prune → Quantize	0.10	0.00

From these initial results, we observe that quantization is highly effective at reducing model size, but it can introduce latency overhead. Pruning, especially structured pruning, shows promise in reducing model complexity without significantly impacting latency.

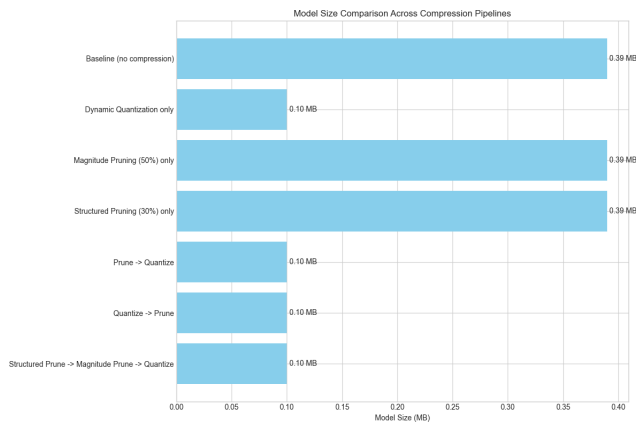


Figure 1: Model Size Comparison Across Compression Pipelines.

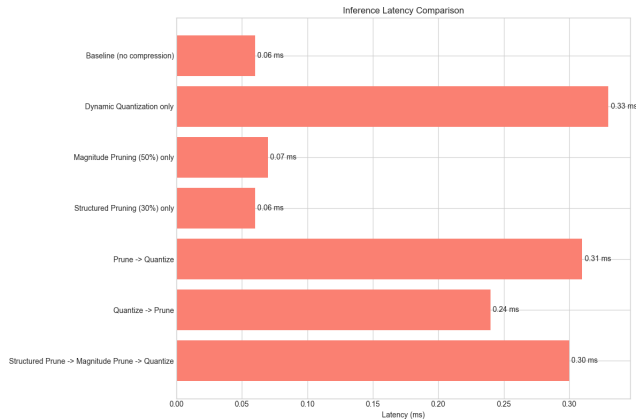


Figure 2: Inference Latency Comparison Across Compression Pipelines.

in this zero-shot setting. The combination and ordering of these techniques lead to interesting trade-offs that warrant further investigation.

2.3 Unit Testing

We have implemented a comprehensive suite of unit tests using pytest to ensure the correctness of our compression components and evaluation metrics. All 20 tests are currently passing, confirming the reliability of our implementation.

```
===== test session starts
=====
...
===== 20 passed in 2.80s
=====
```

3 Plan for Final Paper

Our plan for the final paper includes the following steps:

- Literature Review:** Conduct a thorough review of existing literature on model compression and pipeline optimization. Recent work has shown the effectiveness of combining Neural Architecture Search (NAS) with Knowledge Distillation (KD) to automatically discover optimal student architectures that balance performance and efficiency. This “KD-NAS” approach, as detailed by Trivedi et al. (2023) [2], moves beyond manually selecting student models and instead searches a predefined space of architectures to find one best suited for distillation from a given teacher. This is a promising direction for automating and improving the compression process. Vilar Dantas et al. (2024) [3] provide a comprehensive review of model compression techniques, which will serve as a foundational reference for our work.
- Expand Experiments:**
 - Incorporate StaticQuantizer and ZeroShotDistiller into the experimental pipeline.
 - Apply the compression pipelines to a pre-trained model from torchvision (e.g., ResNet-18) to evaluate performance on a real-world task.
 - Measure the impact on model accuracy on a validation dataset (e.g., CIFAR-10).
- Analyze and Visualize Results:** Create plots and tables to clearly present the trade-offs between model size, latency, and accuracy for different compression strategies.
- Future Work – NAS Integration:** As an extension, we will explore integrating a NAS component into our pipeline. This would involve defining a search space of possible student architectures and using a search strategy (e.g., reinforcement learning) to find an optimal student for our ZeroShotDistiller, automating a key part of the pipeline design, as inspired by the work of Merler et al. (2025) [1].
- Write Final Paper:** Structure the final paper with a comprehensive literature review, detailed methodology, thorough experimental results, and a concluding discussion on the findings.

References

- [1] Michele Merler, Aashka Trivedi, Rameswar Panda, Bishwaranjan Bhattacharjee, Taesun Moon, and Avirup Sil. 2025. Neural architecture search of language models using knowledge distillation. US Patent 12,346,665.
- [2] Aashka Trivedi, Takuma Udagawa, Michele Merler, Rameswar Panda, Yousef El-Kurdi, and Bishwaranjan Bhattacharjee. 2023. Neural Architecture Search for Effective Teacher-Student Knowledge Transfer in Language Models. *arXiv preprint arXiv:2303.09639* (2023).
- [3] Pierre Vilar Dantas, Waldir Sabino da Silva Jr, Lucas Carvalho Cordeiro, and Celso Barbosa Carvalho. 2024. A comprehensive review of model compression techniques in machine learning. *Applied Intelligence* 54 (2024), 11804–11844.